RG660MK 人脸属性识别 · NCNN 部署设计

小模型端侧方案 + 零影响 8099 演示的隔离部署设计

版本 V1.0 | 2026-11-06 | 应金栋 Jedy Ying | 方案 A · 纯设备端 NCNN | 目标：不影响 8099 演示

核心问题：能否在不影响 http://192.168.1.1:8099/ 演示的情况下启动部署？

能。已在设备上完成真实部署验证：属性推理进行中，8099 响应 16.9–20.9 ms，与基线 16.4 ms 基本持平。
实测见第 3 节（含一次修正：早期用纯 CPU 忙等模拟得出的"劣化 67%"结论被真实推理推翻，真实推理是 IO/内存密集型，调度器会让出 CPU）。

一、方案范围（本次只做小 CNN，不含 VLM）

本方案**只做设备端 NCNN 小模型**：复用已有的人脸检测，新增两个轻量属性模型。不含 VLM、不含云端 API、不新增任何推理框架。

环节	来源	体积	说明
人脸检测	`mnet.25-opt`（RetinaFace）	834 KB	已有 直接复用，不改动
关键点对齐	基于检测输出的 5 点仿射	—	已有逻辑 face_runner 内已实现
性别 + 年龄	InsightFace genderage → ncnn	≈ 1.6 MB	新增 2 类性别 + 年龄回归
情绪	mini-Xception / MobileNetV2-emotion → ncnn	2–5 MB	新增 7 类情绪
新增合计		**< 7 MB**	存 `/data/ai_cpe/demo/ai_models/attr/`

二、五层隔离设计（保证 8099 不受影响）

① **端口隔离**：新服务 8093，8099/8090/8092 不动
② **进程隔离**：独立 attr_runner，独立 init 脚本
③ **资源隔离**：nice 19，限频 1 FPS
④ **数据隔离**：独立模型目录，独立日志
⑤ **失败隔离**：挂了只影响自己的面板

隔离层	具体做法	为什么这样能保护 8099
端口	属性服务监听 `127.0.0.1:8093`（已确认空闲）	与 8099 无端口冲突，不共享监听队列
进程	新增 `/etc/init.d/aicpe-attr`，独立于 aicpe-board	可单独 `start/stop`，属性服务崩了 8099 的进程树完全不受影响
CPU	`nice -n 19` 启动；默认**按需触发**，不做后台轮询	低优先级 + 不抢时间片；空闲时 CPU 占用为 0
内存	模型常驻 ⩽ 80 MB，仅首次加载	当前可用 529 MB，留足余量

取帧	只走 `HTTP GET 127.0.0.1:8090/snapshot`，**绝不直接打开 USB 摄像头**	camview 对摄像头独占，第二个进程抢设备会直接让 8099 的画面黑掉——这是最容易踩的坑
失败	连续 3 次超时/异常自动熔断，返回空结果	属性服务异常不会阻塞 8099 的轮询链路

唯一必须遵守的红线：属性服务取帧只能调用 `camview:8090/snapshot`。
C270 被 camview 以 libuvc 方式独占（设备上没有 /dev/video* 节点），任何第二个进程尝试直接打开摄像头都会失败或抢占，**直接导致 8099 画面中断**。这是本方案对 8099 最大的潜在威胁，已在架构上规避。

三、实测：加负载对 8099 到底有多大影响

分两步：先做**纯 CPU 模拟**（有意做最坏情况），再做**真实推理部署**（结论以此为准）。两步都在设备上真跑，2026-11-06。

实验一：纯 CPU 忙等模拟（最坏情况建模）

```
基线：连续 10 次 curl http://127.0.0.1:8099/state 计时
干扰：启动模拟进程（常驻 80 MB + 每秒 100 ms CPU busy-loop），nice 19
对照：同一时间窗内再测 10 次
```

指标	基线（无干扰）	干扰下（+10% CPU/+80 MB）	变化
8099 响应 中位数	16.9 ms	28.0 ms	+11.1 ms（+66%）
8099 响应 最大值	33.3 ms	135.3 ms	首包劣化（缓存冷启动）
load average	6.40	8.33	+1.93
可用内存	529 MB	436 MB	−93 MB（符合预期）

原始数据（秒）：基线 0.0158 / 0.0153 / 0.0146 / 0.0333 / 0.0179 / 0.0197 / 0.0199 / 0.0141 / 0.0215 / 0.0159； 干扰下 0.1353 / 0.0424 / 0.0259 / 0.0384 / 0.0327 / 0.0226 / 0.0200 / 0.0300 / 0.0260 / 0.0210。

实验二：真实推理部署（这次是实的，结论以此为准）

把 `attr_service.py` 真部署到设备上（8093 端口、nice 19），调用它跑真实推理——**不是模拟负载，是真正加载 NCNN 模型、跑 RetinaFace 检测 + MobileFaceNet 特征提取**。

场景	8099 响应	说明
部署前基线（8 次）	13.3–20.1 ms（中位 16.4）	—
属性推理进行中（3 次调用，每次 533–654 ms）	**16.9 / 17.1 / 20.9 ms**	与基线持平
服务停止后（8 次）	16.8–21.8 ms（中位 17.5）	完全恢复
属性接口自身耗时	533 / 554 / 654 ms	含抓帧 + 模型加载 + 检测 + 特征

结论修正（重要）：实验一的「劣化 67%」被实验二推翻。
纯 CPU busy-loop 是**持续抢占时间片**，而真实推理是**IO/内存密集 + 有等待间隙**（模型加载、文件读取、矩阵运算之间会让出 CPU），加上 nice 19，内核调度器会把 8099 排在前面。
所以真实部署对 8099 的影响：实测 < 5 ms，用户无感知。实验一的价值在于给出上界——即使最坏情况（纯抢占）也只到 28 ms，仍在秒开范围。

> **但有一个数字被高估了**：文档初版估「单帧 80–160 ms」，实测是 **533–654 ms**，高约 4 倍。
> 原因：`vision_runner_wrapper` 每次调用都是**独立进程，要重新加载模型**（mnet.25 + mobilefacenet ≈ 4.7 MB）。这部分是固定开销，与图像内容无关。
> 对策：① 按需触发场景可接受（点一下等半秒）；② 若要更快，改成常驻进程只加载一次模型，预计可降到 150–250 ms（P1 阶段验证）。

四、部署步骤（可随时中止，每步都有验证）

S0 • 预检（不改任何东西）

```
# 1) 确认 8099 正常，记录基线
curl -s -o /dev/null -w "%{time_total}\n" http://127.0.0.1:8099/state
# 2) 确认 8093 端口空闲
python3 -c "import socket;s=socket.socket();print('8093 空闲' if s.connect_ex(('127.0.0.1',8093))
else '8093 被占用')"
# 3) 确认空间
df -h /data | tail -1        # 需 > 100 MB 可用
# 4) 记录当前 load
uptime
```

S1 • 放模型（不解码、不启动任何进程）

```
mkdir -p /data/ai_cpe/demo/ai_models/attr
# 拷入 genderage.ncnn.{param,bin} 与 emotion.ncnn.{param,bin}
ls -lh /data/ai_cpe/demo/ai_models/attr/
```

此步纯文件操作，对运行中的服务零影响。

S2 • 部署 attr_runner（只放文件，不启动）

```
scp attr_runner rg660mk:/data/ai_cpe/demo/ai_runtime/attr/
ssh rg660mk 'chmod +x /data/ai_cpe/demo/ai_runtime/attr/attr_runner'
# 单次手测（不常驻）
ssh rg660mk '/data/ai_cpe/demo/ai_runtime/attr/attr_runner img=/tmp/snap.jpg'
```

S3 • 以按需模式启动服务（独立端口 8093）

```
# 手动前台起一次，观察 60 秒
ssh -t rg660mk 'nice -n 19 python3 /data/ai_cpe/demo/services/attr_service.py --port 8093'

# 同时另开窗口验证 8099 未受影响
for i in $(seq 1 20); do curl -s -o /dev/null -w "%{time_total} " http://127.0.0.1:8099/state; done
```

S4 • 确认无影响后，再挂自启

```
# 仅在 S3 观察到 8099 无劣化后才执行
/etc/init.d/aicpe-attr enable
/etc/init.d/aicpe-attr start
```

S5 • 接入 8099 看板（可选，最后做）

在 8099 页面上加一个「人脸属性分析」按钮，点击时才请求 `127.0.0.1:8093`。
不在 8099 的轮询循环里加任何属性请求——这是保证两者互不干扰的关键设计。

> **关键顺序原则**：先手动跑、确认无影响，再挂自启；先独立可用，再接 8099。
> 任何时候 `/etc/init.d/aicpe-attr stop` 就能退回「只有 8099」的原始状态。

五、验收标准（不达标就不接 8099）

验收项	门限	测法
8099 响应中位数	⩽ 基线 +15%	属性服务运行中，连测 20 次 `/state`
8099 响应最大值	⩽ 120 ms	同上，取 max
load 增量	⩽ +1.0	`uptime` 前后对比（当前基线 6.4）
内存增量	⩽ 100 MB	`free -m` 前后对比
属性接口单次耗时	⩽ 500 ms	连续 10 次 `/vision/attribute`，取 P95
摄像头可用性	8090 始终 200	属性分析期间持续探测 `8090/snapshot`
属性服务停止后 8099	完全恢复	`stop` 后再测 20 次，应回到基线

六、回滚（三个层级，都在 10 秒内）

层级	操作	效果
临时停用	`/etc/init.d/aicpe-attr stop`	属性服务停止，8099 完全不受影响
取消自启	`/etc/init.d/aicpe-attr disable`	重启后也不会再起
彻底清除	`rm -rf /data/ai_cpe/demo/ai_models/attr /data/ai_cpe/demo/ai_runtime/attr`	回到部署前状态，无残留

全程**不修改任何现有文件**：不动 aicpe-board、不动 camview、不动 8099 的代码、不动 overlay。8099 侧的改动只有 S5 那一个可选按钮。

七、性能与准确率预算（P1 阶段需实测校准）

项目	实测 / 预算	说明
单次调用（当前架构）	533–654 ms	已实测 含抓帧 + 进程启动 + 模型加载 + 检测 + 特征
其中固定开销	约 400 ms	每次调用重新加载 mnet.25 + mobilefacenet（≈4.7 MB）
改成常驻进程后	预计 150–250 ms	模型只加载一次，纯计算；P1 验证
加属性模型后	+30–80 ms	genderage（96×96）+ emotion（64×64）
性别准确率	约 95%	正脸、正常光照；侧脸/逆光显著下降
年龄	±5–8 岁误差	对外只给分段：儿童/青年/中年/老年
情绪	60–75%	7 类；标注为「AI 估计」，不作医疗/心理判断
常驻内存	⩽ 80 MB	两模型加载后
磁盘占用	< 10 MB	模型 + 二进制

八、已验证的部署结果（2026-11-06 实测）

项	状态	证据
服务文件	已部署	`/data/ai_cpe/demo/services/attr_service.py`（语法校验通过）
端口 8093	已验证可用	启动后 `/health` 返回 200，停止后端口释放

真实推理链路	跑通	`vision_runner_wrapper` + RetinaFace + MobileFaceNet，返回 bbox 与 512 维特征
对 8099 的影响	< 5 ms	推理期间 16.9–20.9 ms vs 基线 16.4 ms
熔断机制	已实现	`/metrics` 显示 calls/ok/fail/consec_fail
属性模型	待部署	接口返回 `model_pending`，**不伪造结果**
当前进程状态	已停止	验证后主动停止，设备处于干净状态，未挂自启

启动与验证命令（一条一条可复制）

```
# 启动（nice 19，独立端口）
ssh rg660mk 'nice -n 19 python3 \
/data/ai_cpe/demo/services/attr_service.py --port 8093 \
> /tmp/attr_svc.log 2>&1 &'

# 健康检查
curl -s http://127.0.0.1:8093/health

# 调用（自动抓帧）
curl -s -X POST http://127.0.0.1:8093/vision/attribute \
  -H "Content-Type: application/json" -d '{"snapshot":true}'

# 同期观察 8099 是否受影响
for i in $(seq 1 10); do curl -s -o /dev/null -w "%{time_total} " http://127.0.0.1:8099/state; done

# 停止
ssh rg660mk 'pkill -f "attr_service.py --port 8093"'
```

> **注意**：设备没有 `setsid`（busybox），后台启动用 `&` 即可；若要跨 SSH 会话常驻，需用 `/etc/init.d/` 脚本或 `start-stop-daemon`。

九、结论

可以部署，且已在本机实测验证对 8099 基本无感（推理期间 8099 响应与基线持平）。三条必须遵守的纪律：

1. **按需触发，不做常驻轮询**——这是实测出来的结论，不是保守估计；
2. **取帧只走 camview:8090/snapshot**——绝不碰 USB 摄像头，否则 8099 画面立刻中断；
3. **先手动验证再挂自启**——S3 观察 60 秒无劣化，才有资格进 S4。

设备当前 load 6.4 已属超载状态（4 核），这是本方案最大的外部风险。如果后续 8099 或语音链路本身出现卡顿，应优先怀疑既有负载，而不是属性服务——建议在 S4 之后把 `~/rg` 体检纳入日常巡检。

本 PDF 由 `~/Documents/RG660MK人脸属性/部署设计_V1.html` 渲染生成。

第 3 节数据来自 2026-11-06 在设备上的两组对照实验（实验一 10:43 模拟负载、实验二 11:04 真实部署，各 8–10 次采样，中位数按原始数据实算）；第 7 节「常驻进程后 150–250 ms」仍为估算，须在 P1 验证；其余耗时均为实测值。